Topic 5: Ontology and Gene Set Enrichment Analysis

Lab Course Genome-Oriented Bioinformatics

10.02.2025

This report presents GOEnrich, a Java-based tool developed to perform Gene Set Enrichment Analysis (GSEA) on differential expression data using the Gene Ontology. GOEnrich integrates multiple statistical tests, including the hypergeometric test, Fisher's exact test with jackknifing, and the Kolmogorov–Smirnov test, to compute enrichment scores and correct for multiple testing via the Benjamini–Hochberg procedure. In addition to standard enrichment statistics, GOEnrich performs an analysis of the GO Directed Acyclic Graph (DAG), including gene association propagation, shortest path calculation to known standard-of-truth GO terms, and DAG overlap analysis. Benchmarking against established tools such as g:Profiler and WebGestalt show differences in performance while underlining that the results are difficult to compare.

Contents

1 Introduction

High-throughput experiments, such as differential expression analyses, generate extensive candidate gene lists, presenting significant data interpretation challenges. A key strategy for addressing these challenges involves leveraging biological knowledge bases to annotate and functionally characterize the gene lists. In this project, a Java program, `GOEnrich`, is developed to integrate differential expression data with structured biological information, enabling the identification of over-represented functional gene sets.

1.1 Gene Ontology

The Gene Ontology (GO) is a structured, controlled vocabulary that standardizes the description of gene functions[1]. GO organized gene product function in three different domains. Molecular Function (MF) describes the molecular activities of individual gene products, such as a specific kinase. Cellular Component (CC) describes the location of gene products, such as the mitochondria. Biological Process (BP) describes the pathways and larger processes to which gene products contribute, such as transport[2]. A singular GO Term is a specific descriptor within one of these domains. Each term conveys details about a gene product's function, location, or role in a larger biological process. It has a list of genes associated with it and a list of relationships with other GO terms. For example, the "is a" relationship indicates that one term is a subtype of another, or "part of," which shows that a gene product is a component of a larger complex or process[1]. These relationships are organized in a directed acyclic graph (DAG) structure, where nodes represent terms, and edges represent relationships between terms. Relationships in this graph are not symmetric, meaning that if term A is a parent of term B, term B is not a parent of term A. Additionally, the graph is acyclic, meaning that there are no loops in the graph. This structure can help us understand how a set of down or upregulated genes affects a biological process, cellular component, or molecular function.

1.2 Over-representation Analysis

Various bioinformatics methods produce long candidate lists of potentially biologically relevant genes. Over-representation analysis (ORA) is a method used to determine whether a particular set of genes or proteins is represented more than expected by chance in a given gene set[3]. ORA requires three inputs:

1. A collection of pathways i.e. GO
2. Genes of interest, which are genes that are differentially expressed significantly (from a differential expression analysis)
3. A background set of genes, which are all genes that were measured in the experiment

P values are calculated using a right-tailed Fisher's test, also known as a cumulative hypergeometric probability. The calculated p-values of a GO Term represent the probability of seeing an intersection the size of the observed intersection or larger[4]. The P-value formula is as follows:

$$P(X \geq k) = 1 - \sum_{i=0}^{k-1} \frac{\binom{M}{i}\binom{N-M}{n-i}}{\binom{N}{n}}$$

Where:

- N is the total number of genes in the background set
- M is the total number of genes in the background set that are annotated with the GO term
- n is the total number of genes in the gene set
- k is the number of genes in the gene set that are annotated with the GO term

[3] The p-values are then adjusted for multiple testing.

1.3 Gene Set Enrichment Analysis

Gene Set Enrichment Analysis (GSEA) is a statistical approach used to determine whether predefined groups of genes (gene sets) show statistically significant enrichment in a ranked list of genes derived from high-throughput experiments, such as differential gene expression analysis. In GSEA, we also use a gene list as an input, except that the gene list is ranked based on a metric representing the gene's differential expression (e.g. log2 fold-change). Attempting to find out if a specific term is enriched, we try to determine whether its genes are randomly distributed throughout the gene list or primarily found at the top or bottom of the list (over/under-represented). The GSEA algorithm calculates an enrichment score (ES) for each gene set, corresponding to weighted Kolmogorov-Smirnov statistics[5]. For this assignment, the Kolmogorov-Smirnov test will be used to estimate the significance without permuting.

2 Materials and Methods

2.1 Program Usage

```
java -jar target/GOEnrichmentRunner.jar
usage: GOEnrichmentRunner [-h] -obo <obo_file> -root <GO_namespace> -mapping <gene2go_mapping_file>
-mappingtype [ensembl|go] -enrich <diffexp_file> -o <output_tsv> -minsize <int>
                           -maxsize <int> [-overlapout <overlap_out_tsv>] [-analysis <analysis_out_tsv>]
named arguments:
  -h, --help             show this help message and exit
  -obo <obo_file>        Path to the OBO file
  -root <GO_namespace>   Specify GO namespace: biological_process, cellular_component,
or molecular_function
  -mapping <gene2go_mapping_file>
                         Path to the gene-to-GO mapping file
  -mappingtype [ensembl|go]
                         Specify the format of the mapping file: ensembl or go
  -enrich <diffexp_file>
                         Path to the enrichment analysis input file
  -o <output_tsv>        Path to the output TSV file for enrichment results
  -minsize <int>         Minimum size of GO entries to consider
  -maxsize <int>         Maximum size of GO entries to consider
  -overlapout <overlap_out_tsv>
                         Optional output file for DAG overlap information
  -analysis <analysis_out_tsv>
                         Optional output file for DAG info
```

2.2 Program Structure

This program calculates enrichment statistics by integrating differential expression data with the hierarchical structure of the Gene Ontology (GO) DAG. It evaluates whether measured genes, particularly those flagged as significantly differentially expressed, are overrepresented in specific GO categories by computing multiple statistical tests—including the hypergeometric test, Fisher's exact test with jack-knifing, and the Kolmogorov-Smirnov (KS) test—and by determining the shortest paths within the GO DAG linking analyzed terms to known truly enriched entries. The main steps of the program are as follows:

1. Parsing of the mapping file
2. Parsing of the OBO file and initialization of the DAG
3. Parsing of the differential expression file
4. Enrichment analysis and calculation of the p-values, FDR, and shortest paths
5. (optional) Calculation of the DAG overlap information
6. Writing of the results to the output file

A `GeneSetEnrichmentAnalysis` object is initialized with the parameters of the analysis, organizes the input data, and stores the enrichment analysis results. Logging is provided at key stages of the program to help debug and analyze the runtime.

2.2.1 Mapping File Parsing

The mapping file must be parsed to extract the gene to GO term associations. Two file formats are supported, depending on the chosen `-mappingtype` parameter. The implementation avoids inefficient methods like `split()` in favor of manual character-by-character parsing, enhancing performance when processing large files.

Ensembl Mapping: For the `ensembl` mapping type, the input is a TSV file (e.g., `goa_human_ensembl.tsv`). The list of GO terms is provided as a concatenated string where individual terms are separated by the delimiter `|`. The parser iterates through each line, first extracting the HGNC symbol and then scanning for GO terms. Each GO term is identified by the `GO:` prefix, the prefix is removed, and the numeric part is parsed into an integer before being added to a set associated with that gene symbol.

GO Mapping: For the `go` mapping type, the input is provided in a compressed GAF file (e.g., `goa_human.gaf.gz`). This file is read directly from its compressed state using a `GZIPInputStream`. The parser processes non-comment lines and focuses on columns 3, 4, and 5. Like the Ensembl parser, the GO term is extracted by manually scanning the string, and its numeric portion is stored in a set mapped to the gene.

Data Strucutre: In both cases, the associations are stored in a `HashMap` where each key is a gene identifier (specifically, the HGNC symbol) and the corresponding value is a `HashSet` of integer representations of GO terms. Integers are used instead of a string representation due to hashing efficiency.

2.2.2 DAG Initialization

The initialization of the DAG begins with parsing the OBO file via the `OboParser`. This parser reads the GO definitions and constructs a hash map of `GOTerm` objects, each encapsulating a unique numeric identifier, a full identifier string, a descriptive name, and placeholders for parent and child relationships. To efficiently capture the hierarchy, temporary parent IDs are stored during parsing, leaving the creation of parent-child links until all terms are loaded. Subsequently, the `resolveParents` method loops over every GO term, using its temporary parent IDs to look up real parent terms in the entries map. For each valid parent found, it links the term as a child to that parent and vice versa. Terms with no temporary IDs are designated as the root, and finally, the temporary IDs are cleared. In parallel, the `resolveLeaves` method identifies leaf nodes (have no children). The DAG is further processed by integrating gene associations from the mapping file through the `resolveGenes` method, which links genes to their corresponding GO terms. The `propagateGenes` method walks through the DAG from the root downwards and, for each node, adds all the genes from its children into its own gene set, iterating through the graph in a postorder fashion. In short, it makes sure that every parent's gene list includes all genes from its descendant terms.

2.2.3 Differential Expression File Parsing

The differential expression file parsing implementation is designed with efficiency in mind. Lines beginning with `#` are used to extract Standard of Truth (SOT) GO terms, and the remaining lines are parsed manually by iterating over characters to extract the gene ID, $\log_2$ fold change value, and significance flag. This approach avoids the overhead of methods like `split()` and stores each parsed record in a hash map for quick access during analysis.

2.2.4 Enrichment Analysis

The enrichment analysis is applied only to GO terms whose associated gene counts fall within the specified minimum and maximum thresholds. The analysis proceeds in two phases: raw enrichment statistics are computed in parallel for each qualifying GO term and the resulting p-values are adjusted using the Benjamini–Hochberg procedure to control the False Discovery Rate (FDR).

Raw Enrichment Statistics Computation Each GO term is processed concurrently using parallel streams. For every term, the following statistics are calculated:

- **Size and Overlap:** The number of measured genes associated with the GO term is determined (size), and the number of those that are significantly differentially expressed (overlap) is recorded.
- **Hypergeometric Test:** To assess the likelihood of the observed overlap occurring by chance, the hypergeometric test is performed with the following parameters:
  - N: The total number of differentially expressed genes in the root (i.e. the background population size).
  - K: The number of significantly differentially expressed genes in the root.
  - n: The number of measured genes associated with the GO term (i.e. the term's *size*).
  - k: The number of significantly differentially expressed genes within the GO term (i.e. the observed *overlap*).
- **Fisher's Exact Test (with Jackknife):** In addition to the hypergeometric test, Fisher's exact test is performed with jackknife resampling. The inputs for this test are analogous to those used for the hypergeometric test, but each parameter is reduced by one to account for the jackknife correction:
  - $N - 1$: Total number of differentially expressed genes in the root minus one.
  - $K - 1$: Number of significantly differentially expressed genes in the root, minus one.
  - $n - 1$: GO term *size* minus one.

  - $k - 1$: Observed *overlap* minus one.

- **Kolmogorov–Smirnov (KS) Test:** The KS test is used to compare the distribution of $\log_2$ fold changes of genes within the GO term (the measured distribution) against that of the background (i.e., the $\log_2$ fold changes of genes present in the root but not in the GO term). In this test:
  - The *measured distribution* is constructed from the fold changes of the genes associated with the GO term.
  - The *background distribution* is built from the fold changes of the remaining genes in the root.

Shortest Path Calculation to a Standard of Truth (SOT) GO Term The algorithm computes the minimal path to the nearest SOT term within the GO hierarchy for GO terms not flagged as Standard of Truth. This is achieved via a bidirectional search:

1. **Upward Search:** A breadth-first search (BFS) is initiated from the analyzed term using parent links. An `ArrayDeque` is used as the queue for BFS operations. Unlike a fixed-size array, the `ArrayDeque` is a resizable double-ended queue that allows efficient insertion and removal at both ends—ideal for BFS where elements are frequently enqueued and dequeued[6]. During this phase, distances (i.e., the number of edges) to each ancestor are recorded in a `HashMap`, along with pointers to reconstruct the upward path.
2. **Downward Search:** For each candidate ancestor (sorted by increasing upward distance), a downward BFS is performed using child links to locate a SOT term. Like the upward search, an `ArrayDeque` manages the queue, while a `HashMap` tracks distances and predecessor nodes for the downward traversal. This dual search (upward and downward) ensures that both ancestral and descendant directions are explored to identify the minimal connecting path.

Once a candidate yielding the smallest combined distance is found, the upward path (from the analyzed term to the candidate) and the downward path (from the candidate to the SOT term) are reconstructed using the stored predecessor pointers. The final path is then formatted as a sequence of GO term names, with the turning point (least common ancestor) marked by an appended asterisk.

P-value Adjustment via Benjamini–Hochberg Correction After raw p-values for the hypergeometric, Fisher's exact, and KS tests are computed, the Benjamini–Hochberg procedure is applied to each set to control the FDR, defined as the expected proportion of false positives among the results declared significant. The adjustment process involves:

1. Sorting the raw p-values in ascending order.
2. Adjusting each p-value by multiplying it by the total number of tests and dividing by its rank (position in the sorted array).
3. Assigning these adjusted p-values back to the corresponding GO term entries.

This correction ensures that the significance levels remain reliable despite the multiple hypothesis tests performed.

Statistical Definitions:

- **Hypergeometric Test:** Evaluates the probability of drawing the observed number of significant genes from a finite population without replacement.
- **Fisher's Exact Test:** Assesses the association between gene significance and GO term membership using contingency tables. With the incorporated jackknife resampling, this test provides a robust estimate of whether the proportion of significantly expressed genes within the GO term is higher than expected by chance—serving as the measure of overrepresentation[7].
- **KS Test:** A nonparametric test that compares the distribution of $\log_2$ fold change values for genes in the GO term against the background distribution. In this context, the KS statistic acts as an enrichment score by quantifying the extent to which the gene set's distribution deviates from the overall background[5].
- **False Discovery Rate (FDR):** The expected fraction of false positives among the significant results after adjusting for multiple hypothesis tests.

2.2.5 Overlap Analysis

The overlap analysis quantifies how two GO terms are related by computing the number of shared genes, the relationship, the shortest path length between the terms, and the maximum overlap percentage. If two terms share one or more genes, the following metrics are calculated:

Pairwise Comparison and Shared Gene Count The program iterates over all unique pairs of filtered GO terms. If two terms share a gene, the following metrics are calculated:

Hierarchical Relationship and Shortest Path Calculation For overlapping pairs, two key metrics are derived to capture their hierarchical relationship:

- **Relative Relationship:** A GO term is deemed relative to another if one is an ancestor of the other. This is established by generating an ancestor map for each term (via a breadth-first search using parent links) and checking if one term is present in the ancestor map of the other.
- **Shortest Path Length:** The shortest path between two GO terms is computed by:
  1. **Upward Search:** An ancestor map is generated for each term, which records every visited ancestor along with the number of edges (upward moves) required to reach it. This is achieved using a BFS implemented with an `ArrayDeque` as the queue.
  2. **Intersection and Distance Calculation:** The two ancestor maps are intersected to identify common ancestors. The total distance is computed for each common ancestor as the sum of the upward moves from each term. The minimal total distance among these is taken as the shortest path length between the two GO terms.

Overlap Metrics In addition to the hierarchical measurements, the analysis computes:

- **Number of Overlapping Genes:** The count of genes shared between the two GO terms.
- **Maximum Overlap Percentage:** Calculated as:

$$\text{max\_ov\_percent} = \frac{\text{num\_overlapping}}{\min(\text{size of term1}, \text{size of term2})} \times 100,$$

  this metric represents the relative degree of overlap based on the smaller gene set. The Jaccard Index could also be used, which instead quantifies similarity based on the intersection divided by the union of the two gene sets.

Implementation and Output The overlap analysis is implemented using parallel processing to handle a large number of GO term pairs efficiently. A global cache is used to store computed ancestor maps, reducing redundant calculations during pairwise comparisons. The first time a term is processed, the ancestor map is calculated and stored in the global cache. If another thread tries to access the ancestor map before the first thread has finished calculating it, it briefly waits and only retrieves the finished map, thereby eliminating thread-related inconsistencies where an incomplete map is returned.

2.3 Input File Format

The program requires three input file types: the OBO, mapping, and differential expression files.

OBO File: The OBO file contains the definitions of Gene Ontology and its relationships. It is parsed to initialize the GO Directed Acyclic Graph (DAG) by skipping obsolete entries and using the `is_a` relationships to build the hierarchical structure.

Mapping File: The mapping file associates genes with GO terms and supports two formats based on the `-mappingtype` parameter:

- **go:** Provided in the Gene Annotation File (GAF) format, the file (e.g., `goa_human.gaf.gz`) is compressed as a `.gz` file. Only non-comment lines (those not starting with `!`) are processed using columns 3–5, which correspond to the gene identifier, the association qualifier modifier, and the GO category identifier. Only associations without any association qualifier modifier are considered.
- **ensembl:** Provided as a tab-separated values (TSV) file (e.g., `goa_human_ensembl.tsv`), it consists of three columns: the Ensembl gene ID, the HGNC gene symbol, and a list of associated GO terms (from all namespaces). In this format, the list of GO terms is separated by the symbol `|`.

Differential Expression File: Contains the experimental results for enrichment analysis and has three columns (TSV):

- **id:** The gene identifier.
- **fc:** The $\log_2$ fold change value represents the gene's differential expression. This value is used in the Kolmogorov-Smirnov (KS) test to compare the distribution of fold changes.
- **signif:** A boolean value indicates whether the gene is significantly differentially expressed. Genes with `signif=true` are used for the over-representation analysis.

Optionally, lines starting with the symbol `#` in the differential expression file indicate simulated truly enriched GO identifiers, serving as a standard of truth.

2.4 Output File Format

The main output file is a tab-separated values (TSV) file containing the following columns:

- **term**: The unique GO identifier (e.g., `GO:1902554`) for the analyzed GO entry.
- **name**: The descriptive name of the GO entry.
- **size**: The number of measured genes associated with the GO entry (i.e., the count of genes both present in the differential expression file and linked to the GO term via the provided mapping).
- **is_true**: A boolean flag indicating whether the GO term is part of the simulated, truly enriched set as specified in the input (lines starting with `#`).
- **noverlap**: The number of significantly differentially expressed genes (where `signif=true`) associated with the GO entry.
- **hg_pval**: The p-value for enrichment computed using the hypergeometric distribution.
- **hg_fdr**: The Benjamini-Hochberg corrected (FDR-adjusted) p-value corresponding to the hypergeometric test.
- **fej_pval**: The enrichment p-value derived from Fisher's exact test, implemented with a jack-knifing approach.
- **fej_fdr**: The FDR-adjusted p-value for Fisher's exact test.
- **ks_stat**: The KS test statistic comparing the log2 fold change distribution of genes associated with the GO term to that of the background gene set.
- **ks_pval**: The p-value resulting from the KS test.
- **ks_fdr**: The FDR-adjusted p-value for the KS test.
- **shortest_path_to_a_true**: For GO entries that are not part of the true set, this column contains a '|' separated list of GO entry names that represent the shortest path in the DAG from the analyzed GO term to the nearest true GO term. Within this path, the least common ancestor (LCA) is annotated with an asterisk (e.g., `regulation of immune system process*`). This field remains empty if the analyzed term is already a true entry or if no true entries are provided.

If the optional `-overlapout` parameter is provided, an additional TSV file containing detailed information about overlapping GO entries that share mapped genes is generated. This file includes the following columns:

- **term1**: The GO identifier (e.g., `GO:1902554`) for the first overlapping DAG entry.
- **term2**: The GO identifier for the second overlapping DAG entry.
- **is_relative**: A boolean value indicating whether the GO entry corresponding to `term1` is an ascendant or descendant of the one corresponding to `term2`. A value of `true` denotes a direct ancestral or descendant relationship, while `false` indicates no such relationship.
- **path_length**: The length of the shortest path between the two GO entries, defined as the minimal number of edges connecting them within the GO DAG.
- **num_overlapping**: The number of gene identifiers that are associated with both GO entries.
- **max_ov_percent**: The maximum percentage (a float value between 0.0 and 100.0) of the shared gene identifiers relative to the total number of genes associated with either `term1` or `term2`.

This overlap output file includes only GO entry pairs that meet the defined `minsize` and `maxsize` criteria and have shared genes.

2.5 Provided Files

The following files were provided for testing the program:

- **simul_exp_go_bp_ensembl.tsv:** Differential expression file for the Biological Process (BP) ontology with simulated, truly enriched standard-of-truth GO Terms.
- **go.obo:** The Gene Ontology OBO (Release 2016-12-24) file contains GO definitions and relationships.
- **goa_human.gaf.gz:** The Gene Annotation File (GAF) for human genes, compressed as a `.gz` file.
- **goa_human_ensembl.tsv:** The Ensembl gene-to-GO mapping file for human genes.
- **simul_exp_go_bp_ensembl_min50_max500.enrich.out:** The expected output file for the provided differential expression file using the Ensembl mapping and the BP ontology with a gene size range of 50 to 500.

3 Results

`GOEnrich` passes the submission server with 100% in 42 seconds.

3.1 Runtime

The program was run for the provided test files on a PC with a Ryzen 5 3600 and 16GB of RAM, using both the Ensembl and GO mapping files for each of the three GO namespaces. The program was executed with a size range of 50 to 500 genes per Term. This filtering helps avoid the issues associated with extremely small gene sets (which can yield inflated scores) and overly large ones (which may lead to inaccurate normalization)[8]. Table 1 shows the runtime breakdown for each step.

The mapping file processing shows considerable differences between the GO and Ensembl. For example, in the Biological Process (BP) ontology, the GO mapping takes 798 ms compared to 181 ms for Ensembl. This pattern is consistent across the Molecular Function (MF) and Cellular Component (CC) tests, suggesting that the zipped GO mapping is more complicated to process. The graph loading operations (loading entries, resolving parents, leaves, genes, and propagating genes) contribute significantly to the runtime. These values differ between ontologies (e.g., 637 ms for GO-BP versus 303 ms for GO-MF) likely due to the varying DAG sizes and complexities. The enrichment calculation step is the most time-consuming. In the BP runs, it takes 3789 ms for GO mapping and 3207 ms for Ensembl. The overlap analysis takes between 1484 and 1908 ms. These two steps dominate the runtime and could be further optimized. However, they are inherently time-consuming by the nature of what they are processing, like the shortest path and the KS test. Loading differential expression records, adjusting p-values, and writing the output requires minimal time (all under 50 ms)

Overall, the total runtimes vary between 1503 ms (Ensembl, CC) and 7217 ms (GO, BP). The enrichment and overlap calculations are the largest contributors to the execution time.

Table 1: Runtime Breakdown for GO/Ensembl Mappings Across GO Namespaces on a Ryzen 5 3600 (16GB RAM) System. Tests used GO entry size constraints of 50 (min) and 500 (max).

Mapping	Ontology	Mapping (ms)	DAG (ms)	DiffExpr (ms)	Enrichment (ms)	p-adjust (ms)	Output (ms)	Overlap (ms)	Total (ms)
GO	BP	798	637	17	3789	15	38	1908	7217
Ensembl	BP	181	725	17	3207	12	38	1484	5677
GO	MF	1093	303	28	1076	5	19	391	2932
Ensembl	MF	155	279	26	843	5	21	364	1705
GO	CC	854	188	17	726	5	18	305	2118
Ensembl	CC	229	229	24	704	4	20	287	1503

To complement this initial analysis, Figure 1 presents a more detailed evaluation of the runtime performance with respect to the effects of parallelization. While Table 1 provides an overall breakdown of the runtime for each processing step, the figure focuses specifically on how parallelization influences the enrichment and overlap calculations, which are the primary bottlenecks. This secondary analysis helps illustrate the significant performance improvements achieved by parallelization across the different GO namespaces and mappings.

3.2 Gene Ontology Namespace Analysis

Various graph metrics can be explored to quantify how the GO Namespaces differ. A `-analysis` parameter was added to the program that calculates and outputs the following metrics:

- The number of gene sets

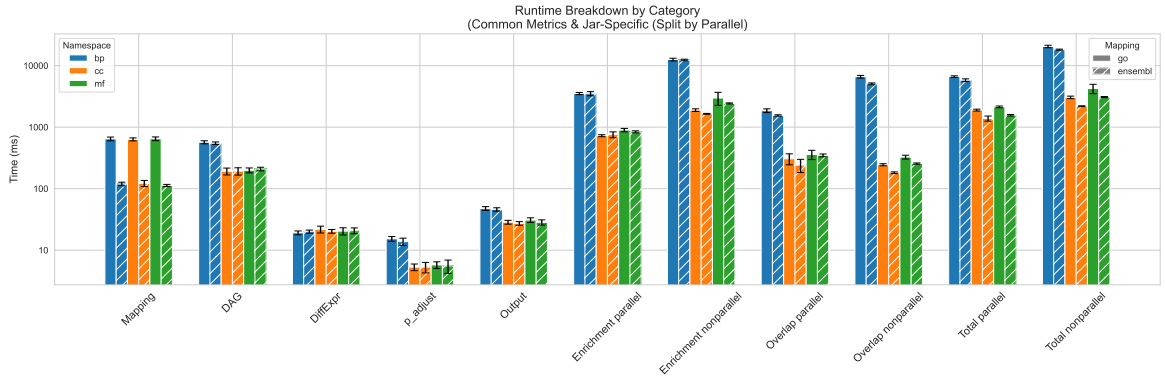


Figure 1: Detailed runtime breakdown for GO/Ensembl mappings across GO namespaces, measured with parallelization enabled and disabled on a Ryzen 5 3600 (16GB RAM) system. Each configuration was executed 10 times, and the results were averaged, with error bars representing the standard deviation. Parallelization affects only the enrichment calculation, overlap calculation, and, by nature, the total runtime. These components are presented separately for parallel and non-parallel runtimes, while the remaining metrics are averaged over both settings. The y-axis is displayed on a $\log_{10}$ scale. It is clear that the parallelization optimization is highly effective, considerably reducing the runtime across all namespaces and mappings.

- The number of total genes (in the root)
- The number of leaf nodes
- The shortest/longest path length from the root to a leaf node
- The number of shortcuts in the DAG: For each term (except the root), each parent edge is a shortcut if the parent's distance to the root + 1 does not equal the term's distance to the root. (See Figure2)
- Distribution of gene set sizes
- Distribution of leaf to root path lengths,
- Distribution of set differences: The number of genes in the parent set that are not in the child set.

These general metrics were calculated for each GO Namespace and mapping type and summarized in Table 2. Since only the number of genes varies between different mappings, the rows for different mappings were combined and a column for the size for each mapping was added.

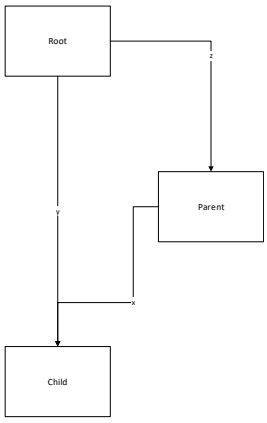
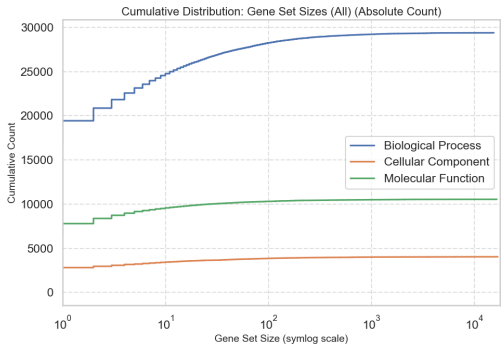


Figure 2: Diagram defining what a shortcut is. If $y < z + x$, with $x = 1$ as it is a child, y is a shortcut.

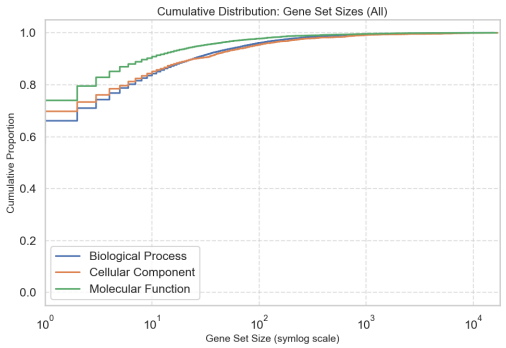
The BP namespace has a substantially higher number of gene states compared to MF and CC. However, despite BP having more terms, the total gene counts (both GO and Ensembl) are similar to CC, possibly suggesting that BP has a higher specificity of its associated genes, as it does not have a proportional increase in the number of genes. Expectedly, the number of shortcuts in the DAG is higher in BP, as it has more terms and leaf nodes. Figure 3 presents the cumulative distribution of gene set sizes across all three GO namespaces. The left subplot shows the absolute cumulative count, while the right subplot displays the relative proportion (CDF). The x-axis is symlog-scaled to accommodate the wide range of values. The BP namespace exhibits a

Table 2: Comparison of various graph metrics for the three Gene Ontology namespaces. **Ontology** indicates the GO Namespace. **Gene Sets** is the total number of terms in the namespace. **Genes (GO)** and **Genes (Ensembl)** denote the total number of genes annotated to the root term using GO and Ensembl mappings, respectively. **Leaf Nodes** represents the number of terminal nodes (terms with no children) in the ontology graph. **Shortest Path** and **Longest Path** show the minimum and maximum number of edges from the root to any leaf node, respectively. **Shortcuts in DAG** counts the parent-child relationships that are classified as shortcuts (i.e., where the parent's distance from the root plus one does not equal the child's distance).

Ontology	Gene Sets	Genes (GO)	Genes (Ensembl)	Leaf Nodes	Shortest Path	Longest Path	Shortcuts in DAG
BP	29385	17026	15497	14991	2	12	13553
MF	10543	17073	16547	8463	1	11	1237
CC	4045	18319	16932	3062	1	8	1172



(a) Absolute Count.



(b) Relative Proportion (CDF).

Figure 3: Comparison of cumulative distributions of gene set sizes. The x-axis is symlog-scaled, which applies a logarithmic transformation to larger values while retaining a linear scale near zero. The BP namespace has a higher cumulative count due to its larger graph. Interestingly, the gene set size distribution remains similar, as seen in the relative distribution.

higher cumulative count due to its larger graph size. However, despite this difference, the distribution of gene set sizes remains similar across all three ontologies. In Figure 4, the gene sets are restricted to the default minimum and maximum sizes (50 and 500, respectively). The distribution is almost identical for all ontologies. Figure 5 displays each GO namespace's cumulative distribution of leaf-to-root path lengths. The CC namespace

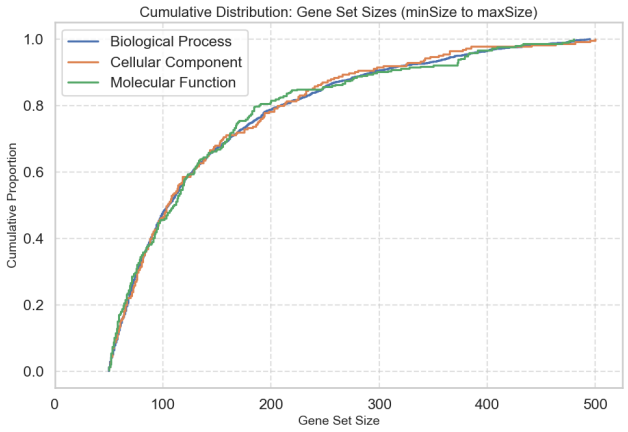


Figure 4: Cumulative Distribution of Gene Set Sizes only for the Minimum (50) to Maximum (500) Size Range. The distribution looks extremely similar for all three ontologies.

has significantly shorter paths than BP and MF due to its smaller graph size. The distribution of path lengths is similar for BP and MF, with the majority of terms having a path length of 4–8 steps. Figure 6 shows each GO

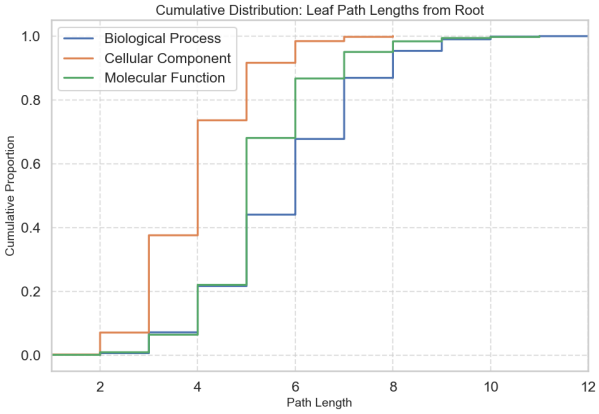


Figure 5: Cumulative distribution of all the leaf-to-root path lengths. CC has significantly shorter path lengths due to its smaller graph size.

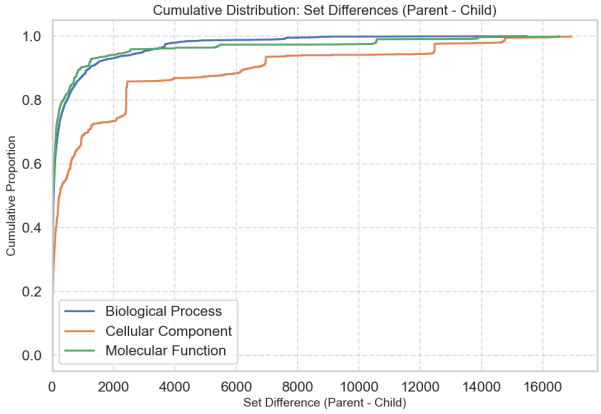


Figure 6: Cumulative Distribution of Set Differences (Parent-Child). BP and MF share a similar distribution, while CC has a higher number of set differences.

namespace's cumulative distribution of set differences (parent-child). The BP and MF distributions are closely aligned, while CC demonstrates a higher number of set differences. This difference may be attributed to the fact that cellular components often aggregate many smaller structures into larger ones, leading to more complex parent-child relationships.

3.3 Comparison to g:Profiler

In order to gauge the performance of `GOEnrich`, g:Profiler[9], a widely used enrichment analysis tool, was applied to the genes marked as significant in the differential expression input file. The analysis was restricted to the GO BP namespace because the simulated input contained standard-of-truth GO terms in this namespace. The search[1] was further limited to GO terms with sizes between 50 and 500 genes. The Benjamini-Hochberg adjusted p-values from g:Profiler were used for comparison. `GOEnrich` was run with the ensembl mapping and BP namespace, with a minsize of 50 and a maxsize of 500. In order to see how `GOEnrich` would perform without the usual size restriction, it was also run with minsize set to 10. The fej_fdr column from `GOEnrich` was used for comparison.

[1] https://biit.cs.ut.ee/gplink/l/ahcyWanp-QS version:e112_eg59_p19_25aa4782

For the Receiver operating characteristic (ROC) analysis, each reported GO term is assigned a continuous score defined as

$$\text{score} = -\log_{10}(\text{FDR}),$$

for our tool, and analogously

$$\text{score} = -\log_{10}(\text{adjusted p-value}),$$

for g:Profiler. This transformation converts low (i.e., more significant) FDR or p-values into high scores so that stronger evidence (more significance) for enrichment corresponds to higher scores. By 'iterating' over the continuous scoring thresholds, the ROC curves are generated that plot the true positive rate (TPR) against the false positive rate (FPR). The Area Under the Curve (AUC) is then calculated to quantify the overall performance of each tool.

It is important to note that, due to filtering differences, only the GO terms actually reported by each method are considered in the ROC analysis. In particular, some SOT terms that fall below the minimum size cutoff in `GOEnrich` (e.g., IDs 0098743, 0098754, 0001906, 1900047, 2000242, 0043473, 0034260) are not included in its output and therefore are not counted as false negatives. There are also some terms that do not exist in the ensembl mapping. Similarly, g:Profiler employs a different ontology version, so certain SOT terms[2] are absent from its output. Consequently, the ROC analysis is based solely on the reported terms, and any SOT term not present in a given output is excluded from that tool's evaluation.

Two thresholds are annotated on the ROC curves:

1. A fixed significance threshold corresponding to a p-value (or FDR) of 0.05. After transformation, this threshold is
$$-\log_{10}(0.05) \approx 1.30.$$
2. The SOT threshold is defined as the minimum score necessary to correctly classify all SOT terms (that aren't filtered).

Figure 7 shows that `GOEnrich`(50) achieved an AUC of 0.94, while g:Profiler achieved an AUC of 0.96. The run with `GOEnrich`(10) achieved a slightly lower AUC of 0.93, as it includes far more significant genes. This is in line with observations made earlier that lowering the minsize parameter will yield more significant terms but also more false positives. The ROC curves are annotated with the fixed threshold (score ≈ 1.30) and the SOT thresholds (score = 7.22 for our `GOEnrich`(50), 24.06 for g:Profiler, and 4.71 for `GOEnrich`(10)), along with their corresponding TPR and FPR values. In `GOEnrich`(50) the SOT threshold yielded a TPR of 1.00 with an FPR of 0.47, whereas the fixed threshold resulted in a TPR of 1.00 but a higher FPR of 0.72. In contrast, g:Profiler's SOT threshold provided a TPR of 1.00 with an FPR of 0.20, and its fixed threshold gave a TPR of 1.00 with an FPR of 0.86. This illustrates how a simple threshold of 0.05 is not optimal in this case since both tools find too many false positives. The SOT threshold for `GOEnrich`(10) is the lowest one but still has the best FPR (0.2) for a TPR of 1. In contrast, the fixed threshold (score ≈ 1.30) yields a TPR of 1.00 across all methods; however, the false positive rate is substantially higher for g:Profiler 0.86 compared to 0.72 for `GOEnrich`(50) and 0.59 for `GOEnrich`(10). The difference in AUC values suggests that g:Profiler is slightly better at distinguishing between true and false positives, but the difference is not substantial. Additionally, due to the differences in file formats and filtering[3], these results are not directly comparable and should be interpreted with caution.

Figure 8 presents an UpSet plot that compares the significant GO terms (FDR or adjusted p-value < 0.05) derived from three analyses: `GOEnrich` with minsize 50, `GOEnrich` with minsize 10, and g:Profiler. The plot shows that all SOT terms were identified by `GOEnrich` (minsize 10). Among these, 7 SOT terms were common to all three methods, 2 terms were uniquely detected by `GOEnrich` (minsize 50) (i.e., not by g:Profiler), and 2 terms were exclusively identified by g:Profiler (i.e., not by `GOEnrich` (minsize 50)). Additionally, `GOEnrich` (minsize 10) found the highest total number of significant genes (2696), followed by g:Profiler (1961) and `GOEnrich` (minsize 50) (1138). These results underscore that while lowering the minsize parameter increases sensitivity by yielding more significant terms, it also introduces more false positives. Thus, even after multiple hypothesis testing corrections, the fixed threshold appears suboptimal because the sheer number of significant terms may render the results less useful.

3.4 WebGestalt Comparison

To assess the GSEA component of `GOEnrich`, it was compared to WebGestalt[10]. The same input files were used, and the analysis was restricted to the BP ontology. The parameters used for WebGestalt are provided in Section 4.2[4]. As with g:Profiler, a custom .GMT file did not work, so the default BP ontology was used.

[2] e.g. GO:0044848, which has been deleted in the current version https://www.ebi.ac.uk/QuickGO/GTerm?id=GO:0044848#term=history

[3] It was attempted to use g:Profiler with the provided input files by using a `.GAF + .OBO -> .GMT` conversion site, but this ultimately did not work. https://biit.cs.ut.ee/gmt-helper/

[4] Results: https://www.webgestalt.org/results/1739092074/

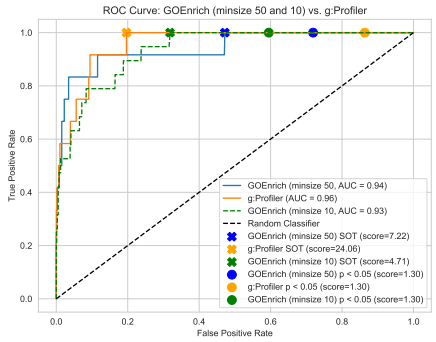


Figure 7: ROC curves comparing `GOEnrich`(minsize 50 and minsize 10) and g:Profiler. The fixed significance threshold (score $\approx$ 1.30) and the SOT thresholds are annotated. Due to differing ontology versions, only the terms reported by both tools are considered. The score is defined as $-\log_{10}$(significance).

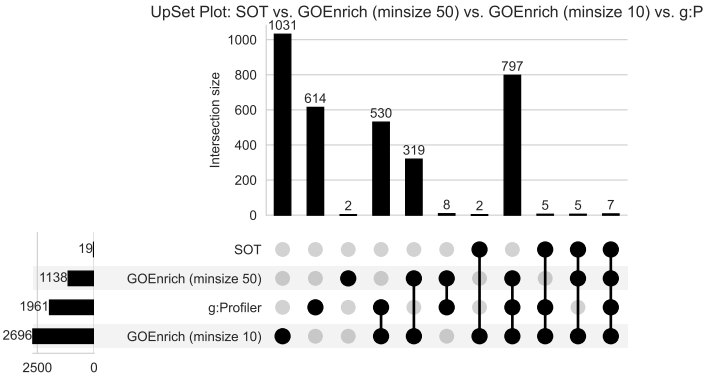


Figure 8: Upset plot comparing SOT terms from significant GO analyses performed using `GOEnrich` (minsize = 50), `GOEnrich` (minsize = 10), and `g:Profiler` (all with FDR/adjusted p-value ¡ 0.05). The diagram shows that all SOT terms were identified by `GOEnrich`(10). Among these, 7 terms were common to all three methods, 2 terms were uniquely detected by `GOEnrich`(50) (i.e., not by `g:Profiler`), and 2 terms were exclusively identified by `g:Profiler` (i.e., not by `GOEnrich`(50)). Additionally, `GOEnrich`(10) found the highest total number of significant genes (2696), followed by `g:Profiler` (1961) and `GOEnrich`(50) (1138).

Unfortunately, this once again makes it difficult to compare our results, as a different ontology version was used. The minsize and maxsize parameters were set to 50 and 500, respectively, to match the `GOEnrich` analysis. The same transformations were done in Section 3.3, meaning only the terms reported by both tools were considered for the ROC curve. The same `GOEnrich` results were used from Section 3.3, except that the ks_fdr column was used for comparison.

Figure 9 shows the ROC curves comparing `GOEnrich` and WebGestalt. The AUC for `GOEnrich` is 0.94, while WebGestalt achieved a higher AUC of 0.97. For `GOEnrich`, the SOT threshold is 0.46, which yields a TPR of 1.00 and an FPR of 0.57; using the fixed threshold (score $\approx$ 1.30) results in a TPR of 0.92 and an FPR of 0.25. In contrast, WebGestalt has a SOT threshold of 1.18 (TPR = 1.00, FPR = 0.20) and, at the fixed threshold (score $\approx$ 1.30), a TPR of 0.88 with an FPR of 0.06. This suggests that while both tools can identify true positives, WebGestalt has a considerably lower FPR at the fixed threshold of (0.05). Overall, these results show that the GSEA method filters out more genes than the ORA method (As seen in Figure 10 and Figure 8). Due to the class imbalance between true and false, a precision-recall curve might be more informative than the ROC curve.

Figure 10 depicts a Venn diagram comparing the significant GO terms (FDR $<$ 0.05) among the SOT set, `GOEnrich`, and WebGestalt. The diagram shows that both tools correctly identified 6 SOT terms, with 5 terms uniquely identified by `GOEnrich` and 1 term uniquely identified by WebGestalt. However, this difference is likely due to the differing ontology versions used by the two tools and the minsize and maxsize parameters, excluding some of the SOT terms smaller than 50. More interestingly, the diagram shows that `GOEnrich` identifies considerably more significant terms than WebGestalt (375 vs 25), showing that WebGestalt is more stringent in its filtering.

4 Supplement

4.1 Supplementary Figures

The supplementary figures provided in the `plots/` folder of the submission directory further analyze the results of our gene set enrichment analysis. The analysis was performed on gene sets filtered to include only those with sizes between 50 and 500. The figures are summarized as follows:

- **Distribution of Significant Genes (SGs):** A histogram (with kernel density estimation) showing the distribution of the number of significant genes (`noverlap`) across the gene sets.
- **Enrichment Score and p-value Distributions:** A multi-panel figure displaying histograms of the enrichment score (KS statistic) and the p-values from the hypergeometric test, Fischer's exact test, and KS test.

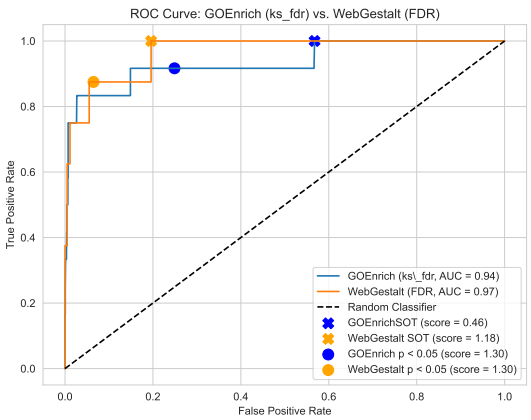


Figure 9: ROC curves comparing `GOEnrich` and WebGestalt. The fixed significance threshold (score $\approx$ 1.30) and the SOT thresholds are annotated. Due to differing ontology versions, only the terms reported by both tools are considered. The AUC for `GOEnrich` is 0.94, while WebGestalt achieved an AUC of 0.97. The score is defined as $-\log_{10}(\text{FDR})$.

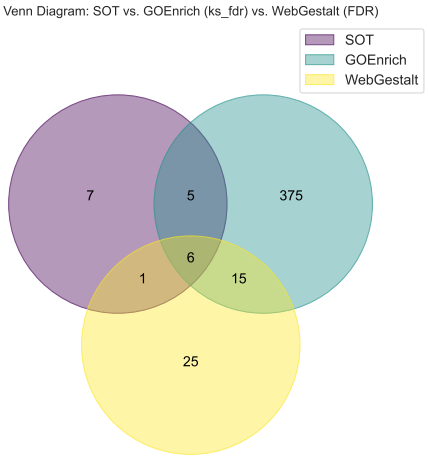


Figure 10: Venn diagram comparing the significant GO terms (FDR/adjusted p-value < 0.05) among the SOT set, `GOEnrich` (ks_fdr), and WebGestalt (FDR).

- **Scatter Plot of KS Statistic vs. Gene Set Size:** A scatter plot illustrating the relationship between the gene set size and the enrichment score (KS statistic).
- **Scatter Plot of KS p-value vs. Gene Set Size:** A scatter plot demonstrating how the KS test p-value varies with gene set size.
- **Scatter Plots of p-value vs. FDR:** For each statistical test (hypergeometric, Fischer's exact, and KS), scatter plots comparing raw p-values against FDR-corrected values are provided. Each plot includes a reference line (i.e., $y = x$) to facilitate visual assessment of the relationship.

These plots offer a comprehensive overview of the key metrics derived from our enrichment analysis.

4.2 WebGestalt Parameters

Enrichment method: GSEA Organism: hsapiens
Enrichment Categories: geneontology_Biological_Process_noRedundant
Interesting list: . ID type: genesymbol
The interesting list contains 17054 user IDs in which 16241 user IDs are unambiguously mapped to 16241 unique entrezgene IDs and 813 user IDs can not be mapped to any entrezgene ID.
The GO Slim summary are based upon the 16241 unique entrezgene IDs.
Among the 16241 unique entrezgene IDs, 13634 IDs are annotated to the selected functional categories, which are used for the enrichment analysis.
Parameters for the enrichment analysis:

Minimum number of IDs in the category: 50
Maximum number of IDs in the category: 500
Significance Level: FDR < 1
Number of permutation: 1000

References

[1] Markus List. "Enrichment Analysis". Unpublished Work. 2022.

[2] *Gene ontology and pathway analysis*. Electronic Book Section. 2023. URL: https://bioinformatics.ccr.cancer.gov/docs/b4b/Module3_Pathway_Analysis/Lesson17/#what-is-gene-ontology (Accessed: 7 Feb. 2025).

[3] Cecilia Wieder et al. "Pathway analysis in metabolomics: Recommendations for the use of over-representation analysis". In: *PLOS Computational Biology* 17.9 (Sept. 2021). Ed. by Kiran Raosaheb Patil, e1009105. DOI: https://doi.org/10.1371/journal.pcbi.1009105.

[4] 2025. URL: https://biit.cs.ut.ee/gprofiler/page/docs#custom_gmt (Accessed: 7 Feb. 2025).

[5] A. Subramanian et al. "Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles". In: *Proceedings of the National Academy of Sciences* 102.43 (Sept. 2005), pp. 15545–15550. DOI: https://doi.org/10.1073/pnas.0506580102. (Accessed: 7 Feb. 2025).

[6] GeeksforGeeks. *ArrayDeque in Java*. Feb. 2023. URL: https://www.geeksforgeeks.org/arraydeque-in-java/ (Accessed: 7 Feb. 2025).

[7] Gergely Csaba. *Gene Set Enrichment*. Provided slides. 2019.

[8] *Gene Set Enrichment Analysis (GSEA) User Guide*. URL: https://www.gsea-msigdb.org/gsea/doc/GSEAUserGuideFrame.html (Accessed: 7 Feb. 2025).

[9] Liis Kolberg et al. "g:Profiler-interoperable web service for functional enrichment analysis and gene identifier mapping (2023 update)". In: *Nucleic Acids Research* 51.W1 (May 2023), gkad347. DOI: https://doi.org/10.1093/nar/gkad347. URL: https://pubmed.ncbi.nlm.nih.gov/37144459/.

[10] John M Elizarraras et al. "WebGestalt 2024: faster gene set analysis and new support for metabolomics and multiomics". In: *Nucleic acids research* (May 2024). DOI: https://doi.org/10.1093/nar/gkae456.

[11] Charles A. Tilford and Nathan O. Siemers. "Gene Set Enrichment Analysis". In: *Protein Networks and Pathway Analysis*. Ed. by Yuri Nikolsky and Julie Bryant. Totowa, NJ: Humana Press, 2009, pp. 99–121. ISBN: 978-1-60761-175-2. DOI: 10.1007/978-1-60761-175-2_6. URL: https://doi.org/10.1007/978-1-60761-175-2_6.

[12] Suzi Aleksander et al. "The Gene Ontology Knowledgebase in 2023". In: *Genetics* 224.1 (Mar. 2023). DOI: https://doi.org/10.1093/genetics/iyad031.

[13] Michael Ashburner et al. "Gene Ontology: tool for the unification of biology". In: *Nature Genetics* 25.1 (May 2000), pp. 25–29. DOI: https://doi.org/10.1038/75556.

[14] T. Beissbarth and T. P. Speed. "GOstat: find statistically overrepresented Gene Ontologies within a group of genes". In: *Bioinformatics* 20.9 (Feb. 2004), pp. 1464–1465. DOI: https://doi.org/10.1093/bioinformatics/bth088.

[15] Jüri Reimand et al. "g:Profiler–a web-based toolset for functional profiling of gene lists from large-scale experiments". In: *Nucleic Acids Research* 35.suppl_2 (July 2007), W193–W200. DOI: https://doi.org/10.1093/nar/gkm226. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1933153/.

[16] Paul D. Thomas et al. "PANTHER: Making genome-scale phylogenetics accessible to all". In: *Protein Science* 31.1 (Nov. 2021), pp. 8–22. DOI: https://doi.org/10.1002/pro.4218.

[17] *Grammarly: Grammar Checker and Writing Assistance*. https://www.grammarly.com. 2025. (Accessed: 7 Feb. 2025).

[18] OpenAI. *ChatGPT [Large language model]*. https://chat.openai.com. 2025. (Accessed: 7 Feb. 2025).